Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Catedrático/a: Erick Marroquín

Laboratorio 4. Polimorfismo. Análisis y Diseño

Javier Alejandro Apen Solis – 26864

Nathalia Isabela Escobar Otto - 261136

06/11/2026

Requisitos Funcionales

RF1. Conservar en memoria los equipos, encuentros y exhibiciones de la copa. Cada equipo registra código, nombre, empresa y puntos de tabla. Cada encuentro conserva deporte, dos equipos, marcador, estado, fecha, hora y sede. Cada exhibición conserva título, estado, fecha, hora y sede; carece de equipos y resultado. La programación y el marcador permanecen sin valor hasta que se registren.

RF2. Registrar equipos con código no vacío y único dentro de la copa, inicialmente con cero puntos de tabla. Crear encuentros entre dos equipos registrados y distintos, eligiendo fútbol sala o baloncesto; nacen sin programar. Crear exhibiciones con título, también sin programar.

RF3. Programar eventos con fecha válida, hora entera entre 0 y 23 y sede identificada. Rechazar coincidencias de sede, fecha y hora con otro evento; en encuentros, rechazar también que cualquiera de sus equipos tenga otro encuentro en esa fecha y hora. Reprogramar encuentros mientras no estén finalizados y exhibiciones programadas. Al comprobar conflictos se excluye el propio evento.

RF4. Registrar una sola vez el resultado de un encuentro programado, mediante dos enteros no negativos. En fútbol sala se admiten empates: victoria 3 puntos de tabla, empate 1 y derrota 0. En baloncesto se rechazan empates: victoria 2 y derrota 1. Un resultado válido conserva el marcador, suma los puntos a ambos equipos y finaliza el encuentro.

RF5. Mostrar todos los equipos en una tabla de posiciones, ordenados por puntos de tabla de mayor a menor y, en caso de igualdad, por nombre alfabéticamente. Los puntos de tabla se acumulan entre deportes y se distinguen del marcador de cada encuentro.

RF6. Mostrar y permitir recorrer el calendario con encuentros y exhibiciones que tengan fecha, incluidos los encuentros finalizados. Ordenarlo por fecha, después por hora y luego por nombre de sede, con un único criterio y sin exponer su almacenamiento interno.

RF7. Mantener disponible el menú de consola hasta salir. Ofrecer registro de equipos, creación de encuentros y exhibiciones, programación, reprogramación, registro de resultados y consulta de tabla y calendario. Rechazar datos inválidos con un mensaje claro, conservar el estado previo y permitir continuar ante entradas con formato incorrecto.

RF8. Cargar al iniciar, directamente desde el código, al menos cuatro equipos de dos empresas, cuatro encuentros con ambos deportes representados y al menos dos resultados, y una exhibición programada. Los datos deben mostrar la puntuación de ambos deportes, un empate en fútbol sala y un desempate por nombre en la tabla. Los datos se conservan únicamente durante la ejecución.

Reglas y comportamiento compartido

Regla o comportamiento	Abstracción responsable y motivo
Validación propia del marcador y puntos de fútbol sala	Clase concreta FutbolSala: admite empates y asigna 3, 1 o 0 puntos según victoria, empate o derrota.
Validación propia del marcador y puntos de baloncesto	Clase concreta Baloncesto: exige marcadores distintos y asigna 2 o 1 puntos según victoria o derrota.
Dos equipos, marcador, estados y registro de resultados	Clase abstracta Encuentro: reúne datos y validaciones comunes, incluidos enteros no negativos, y delega las reglas deportivas a sus subclases.
Recibir resultados y distribuir puntos de tabla	Interfaz Puntuable: define la capacidad de los encuentros para procesar resultados sin exigir conocer el deporte.
Programarse, reprogramarse y aparecer en el calendario	Interfaz Programable: contrato común a Encuentro y Exhibicion. Extiende Comparable<Programable> y concentra el orden cronológico en un método default.
Datos y estados propios de una exhibición	Clase concreta Exhibicion: implementa Programable y conserva título y programación, sin heredar de Encuentro.
Conflictos de sede y de equipos; consultas ordenadas	Clase concreta Copa: dispone de todos los eventos para validar conflictos. Equipo implementa Comparable<Equipo> para ordenar la tabla; Copa implementa Iterable<Programable> para recorrer el calendario.

Los encuentros y las exhibiciones comparten fecha, hora y sede, la posibilidad de cambiar su programación y su presencia en el calendario. El contrato Programable unifica esas operaciones sin obligarlos a compartir una jerarquía de herencia.

Estados y operaciones

Objeto	Estado inicial	Operación válida	Estado resultante
Encuentro	No existe	Crear con dos equipos distintos	SIN_PROGRAMAR
Encuentro	SIN_PROGRAMAR	Programar sin conflictos	PROGRAMADO
Encuentro	PROGRAMADO	Reprogramar sin conflictos	PROGRAMADO
Encuentro	PROGRAMADO	Registrar resultado válido	FINALIZADO
Exhibición	No existe	Crear con título	SIN_PROGRAMAR
Exhibición	SIN_PROGRAMAR	Programar sin conflictos	PROGRAMADA
Exhibición	PROGRAMADA	Reprogramar sin conflictos	PROGRAMADA

Un encuentro FINALIZADO conserva su programación y resultado; rechaza reprogramaciones y nuevos resultados. Un encuentro SIN_PROGRAMAR rechaza resultados. Una exhibición no recibe resultados ni pasa a FINALIZADO. Cualquier operación inválida deja intactos el estado, los datos y los puntos de tabla.

Clases e interfaces propuestas

Todas las clases e interfaces propuestas son públicas. Visibilidad de sus miembros: + public, - private, # protected y ~ acceso de paquete. En los métodos se indica el retorno después de dos puntos; los constructores no tienen retorno. Los atributos son privados para proteger el estado. LocalDate representa la fecha; Integer permite usar null cuando todavía no hay hora o marcador.

Se proponen los enum públicos EstadoEncuentro {SIN_PROGRAMAR, PROGRAMADO, FINALIZADO} y EstadoExhibicion {SIN_PROGRAMAR, PROGRAMADA}. Representan únicamente los estados válidos; no requieren atributos ni métodos propios.

Interfaz Programable

Extiende Comparable<Programable>. Define la programación y consulta de cualquier evento del calendario. No almacena atributos ni tiene constructor.

Miembro y tipo	Propósito
+ programar(LocalDate fecha, int hora, String sede): void	Asigna o cambia la programación si el estado lo permite. Copa verifica antes los conflictos globales.
+ getFecha(): LocalDate + getHora(): Integer + getSede(): String	Consultan la programación; devuelven null mientras el evento esté sin programar.
+ getDescripcion(): String	Describe el evento para seleccionarlo y mostrarlo en el calendario.
+ getEquipos(): List<Equipo>	Devuelve una lista no modificable de los equipos que ocupan ese horario. Una exhibición devuelve una lista vacía.
+ default compareTo(Programable otro): int	Implementa una sola vez la comparación por fecha, hora y sede. Se aplica a eventos con programación; Copa excluye los demás al formar el calendario.

getEquipos() expresa qué equipos quedan ocupados al programar un evento. La lista vacía de una exhibición permite validar conflictos sin preguntar por el tipo de objeto y sin atribuirle equipos ficticios.

Interfaz Puntuable

Define las operaciones disponibles para los encuentros que aportan puntos de tabla. No contiene atributos ni constructor.

Miembro y tipo	Propósito
+ registrarResultado(int marcadorA, int marcadorB): void	Valida el resultado, asigna los puntos de ambos equipos y finaliza el encuentro una sola vez.
+ getDescripcion(): String	Identifica el encuentro al seleccionar dónde registrar el resultado; comparte esta firma con Programable.

Clase concreta Equipo

Representa a una empresa participante y acumula puntos de tabla. Implementa Comparable<Equipo>.

Miembro y tipo	Propósito
- final codigo: String - final nombre: String - final empresa: String	Identificación única, nombre del equipo y empresa representada, respectivamente.
- puntosTabla: int	Total acumulado de puntos de tabla; inicia en 0.
+ Equipo(String codigo, String nombre, String empresa)	Recibe los datos de identificación. Rechaza código vacío; Copa verifica que no esté repetido.
+ getCodigo(): String + getNombre(): String + getEmpresa(): String + getPuntosTabla(): int	Consultan cada atributo sin permitir cambios directos.
~ sumarPuntos(int cantidad): void	Añade puntos no negativos desde el registro de un resultado. Su acceso se limita al paquete del modelo.
+ compareTo(Equipo otro): int	Compara puntos de mayor a menor y, si son iguales, nombre de menor a mayor.

Clase abstracta Encuentro

Implementa Programable y Puntuable. Centraliza los datos, las transiciones y el proceso de registrar un resultado. No puede instanciarse directamente.

Miembro y tipo	Propósito
- final equipoA: Equipo - final equipoB: Equipo	Referencias a los dos equipos participantes; no pueden ser nulos ni tener el mismo código.
- marcadorA: Integer - marcadorB: Integer	Anotaciones de cada equipo en este encuentro; comienzan en null.

Miembro y tipo	Propósito
- estado: EstadoEncuentro	Controla las operaciones permitidas; inicia en SIN_PROGRAMAR.
- fecha: LocalDate - hora: Integer - sede: String	Almacenan la programación común a todos los deportes; comienzan en null.
# Encuentro(Equipo equipoA, Equipo equipoB)	Valida y asigna ambos equipos e inicializa el marcador, el estado y la programación.
+ final programar(LocalDate fecha, int hora, String sede): void	Implementa Programable. Valida fecha, hora y sede; rechaza FINALIZADO y pasa a PROGRAMADO.
+ final registrarResultado(int marcadorA, int marcadorB): void	Implementa Puntuable. Exige PROGRAMADO y marcadores no negativos; consulta las reglas del deporte antes de modificar los datos.
+ getFecha(): LocalDate + getHora(): Integer + getSede(): String	Implementan las consultas de programación de Programable.
+ getEquipos(): List<Equipo>	Implementa Programable: devuelve una lista no modificable con equipoA y equipoB.
+ getDescripcion(): String	Implementa ambas interfaces: muestra deporte, equipos, estado y, si existe, marcador.
# abstract nombreDeporte(): String	Permite identificar el deporte en la descripción común.
# abstract esMarcadorValido(int a, int b): boolean	Determina la validez específica del deporte, después de comprobar que a y b no sean negativos.
# abstract calcularPuntos(int propios, int contrarios): int	Devuelve los puntos de tabla de un equipo según su marcador y el del rival.

registrarResultado calcula primero los puntos de A y de B, invocando calcularPuntos con los marcadores en ambos órdenes. Solo después de todas las validaciones guarda el marcador, suma los puntos y cambia a FINALIZADO. Así, un rechazo no deja cambios parciales. El método es final para conservar este proceso en todos los deportes.

Clases concretas FutbolSala y Baloncesto

Ambas extienden Encuentro y no agregan atributos. Heredan la programación, el estado y el registro del resultado; sobrescriben únicamente los siguientes métodos abstractos.

Miembro y tipo	FutbolSala	Baloncesto
+ Constructor(Equipo a, Equipo b)	FutbolSala(a, b) llama a super(a, b).	Baloncesto(a, b) llama a super(a, b).
# nombreDeporte(): String	Devuelve “Fútbol sala”.	Devuelve “Baloncesto”.
# esMarcadorValido(int a, int b): boolean	Devuelve true para los marcadores no negativos ya validados, incluidos los empates.	Devuelve a != b.
# calcularPuntos(int propios, int contrarios): int	Devuelve 3 si gana, 1 si empata y 0 si pierde.	Devuelve 2 si gana y 1 si pierde; el empate ya fue rechazado.

Clase concreta Exhibicion

Implementa únicamente Programable. Conserva su propia información y usa el compareTo común de esa interfaz.

Miembro y tipo	Propósito
- final titulo: String	Nombre de la clínica o demostración, recibido al crearla.
- estado: EstadoExhibicion	Inicia en SIN_PROGRAMAR y cambia a PROGRAMADA al asignar horario.
- fecha: LocalDate - hora: Integer - sede: String	Fecha, hora y sede de la exhibición; comienzan en null.
+ Exhibicion(String titulo)	Asigna el título e inicializa estado y programación.
+ programar(LocalDate fecha, int hora, String sede): void	Valida fecha, hora y sede; asigna los datos y cambia a PROGRAMADA. Permite repetir la operación para reprogramar.
+ getFecha(): LocalDate + getHora(): Integer + getSede(): String	Implementan las consultas de Programable.
+ getEquipos(): List<Equipo>	Devuelve una lista vacía no modificable, porque la exhibición no ocupa equipos.
+ getDescripcion(): String	Devuelve el título y el estado para identificar y mostrar la exhibición.

Clase concreta Copa

Registra equipos y eventos, comprueba conflictos y ofrece la tabla y el calendario. Implementa Iterable<Programable>.

Miembro y tipo	Propósito
- final equipos: List<Equipo>	Conserva los equipos registrados.
- final eventos: List<Programable>	Conserva todos los eventos, estén o no programados.
- final encuentros: List<Puntuable>	Conserva referencias a los encuentros que aceptan resultados. Son los mismos objetos incluidos en eventos; no se duplican sus datos.
+ Copa()	Inicializa las tres colecciones como ArrayList vacíos.
+ registrarEquipo(String codigo, String nombre, String empresa): Equipo	Valida el código no vacío y no repetido, crea el equipo y lo registra.
+ buscarEquipo(String codigo): Equipo	Obtiene un equipo registrado; rechaza códigos inexistentes.
+ agregarEvento(Programable evento): void	Registra un evento no nulo y aún no agregado. Permite incorporar exhibiciones y otros eventos del calendario.
+ agregarEncuentro(Encuentro encuentro): void	Comprueba que sus equipos pertenezcan a la copa. Registra el mismo objeto en eventos y encuentros, una sola vez.
+ programarEvento(Programable evento, LocalDate fecha, int hora, String sede): void	Verifica que el evento esté registrado y valida conflictos antes de llamar a programar.
- validarProgramacion(Programable evento, LocalDate fecha, int hora, String sede): void	Revisa los datos y, a igual fecha y hora, rechaza sede coincidente o equipos compartidos con otro evento. Excluye el propio evento.
+ registrarResultado(Puntuable encuentro, int a, int b): void	Verifica que esté registrado en encuentros y delega el resultado al contrato Puntuable.
+ getEventos(): List<Programable> + getEncuentros(): List<Puntuable>	Devuelven copias no modificables para seleccionar eventos y encuentros desde la consola.
+ getTablaPosiciones(): List<Equipo>	Devuelve una copia ordenada y no modificable de equipos, usando Equipo.compareTo.
+ iterator(): Iterator<Programable>	Filtra eventos con fecha, ordena una copia con Programable.compareTo y devuelve su iterador sin permitir remove. Habilita for-each sobre la copa.

Los encuentros se incorporan mediante agregarEncuentro; las exhibiciones, mediante agregarEvento. Las programaciones se solicitan a Copa para comprobar la disponibilidad global antes de cambiar el objeto. Las listas devueltas no permiten agregar ni quitar elementos del registro interno.

Clase concreta AplicacionArenaGT

Gestiona la interacción de consola y delega las operaciones del dominio a Copa.

Miembro y tipo	Propósito
- final copa: Copa - final entrada: Scanner	Copa administrada y lector de entradas de consola, respectivamente.
+ AplicacionArenaGT()	Crea una Copa vacía y un Scanner asociado a System.in.
+ static main(String[] args): void	Crea la aplicación y llama a ejecutar.
- ejecutar(): void	Carga los datos iniciales y mantiene el menú hasta que el usuario elija salir.
- cargarDatosIniciales(): void	Construye los equipos y eventos exigidos y los registra, programa y puntúa usando las mismas operaciones del modelo.
- atenderOpcion(int opcion): void	Solicita datos, selecciona los objetos, invoca Copa y muestra resultados o errores claros.
- leerEntero(String mensaje): int - leerFecha(String mensaje): LocalDate	Solicitan entradas y repiten la lectura ante formatos incorrectos, sin terminar la ejecución.

Inicialización y colaboración de constructores

Los objetos reciben sus datos iniciales por los parámetros de los constructores descritos. Al crear FutbolSala o Baloncesto, su constructor llama a super(a, b): Encuentro valida los participantes e inicializa SIN_PROGRAMAR, marcador null y programación null. Las subclases no repiten esos atributos ni su inicialización. AplicacionArenaGT crea la copa y carga los datos mediante las operaciones normales del modelo, incluidas la programación y la asignación de puntos. Las interfaces no tienen constructor ni se instancian.

Justificación de las dos interfaces

Programable representa la capacidad de reservar un horario y aparecer en el calendario; Puntuable representa la capacidad de recibir un resultado y asignar puntos de tabla. Un encuentro tiene ambas capacidades; una exhibición solo tiene la primera. Separarlas permite que cada clase cumpla las operaciones que corresponden a su responsabilidad.

Si una exhibición debiera implementar Puntuable, tendría que inventar equipos y resultados o incluir un método que no hiciera nada o lanzara una excepción. Ninguna opción representa su comportamiento. La separación evita esos métodos artificiales y permite procesar resultados mediante Puntuable; después, la tabla ordena los puntos acumulados de los equipos sin conocer el deporte.

Incorporación de un tercer deporte

Para agregar, por ejemplo, voleibol, se crea Voleibol como subclase de Encuentro. Su constructor llama a super(a, b) y sobrescribe nombreDeporte, esMarcadorValido y calcularPuntos con las reglas que se definan para ese deporte. Se agrega la opción correspondiente en la consola para construirlo y registrarlo mediante Copa.agregarEncuentro.

La tabla de posiciones y el calendario no requieren cambios: la nueva subclase hereda Puntuable y Programable. Equipo mantiene la comparación por puntos y nombre; Copa conserva la validación de conflictos y su recorrido ordenado. Las reglas deportivas se incorporan en la nueva subclase, sin comprobaciones de tipo en la tabla ni en el calendario.